

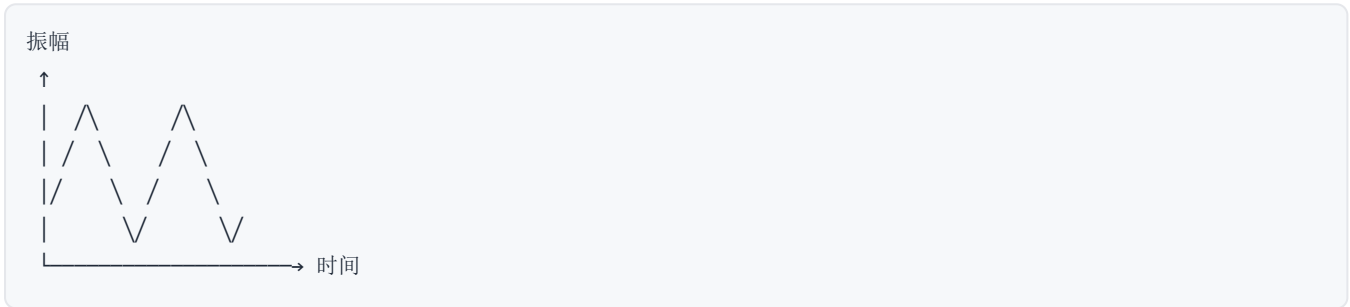
# 技术基础

这篇文档为你补充理解本项目所需的信号处理与深度学习基础知识。大一阶段能接触到这些，会很有帮助。

## 1. 声音的数字表示

### 1.1 波形 (Waveform)

计算机把声音存成一串数字 (sample)：每个数字记录某一瞬间的空气压强 (振幅)。常见的 CD 音质是 **44100 Hz** (每秒 44100 个样本点)。



一首 3 分钟的 CD 歌曲  $\approx 3 \times 60 \times 44100 = 7,938,000$  个样本点。

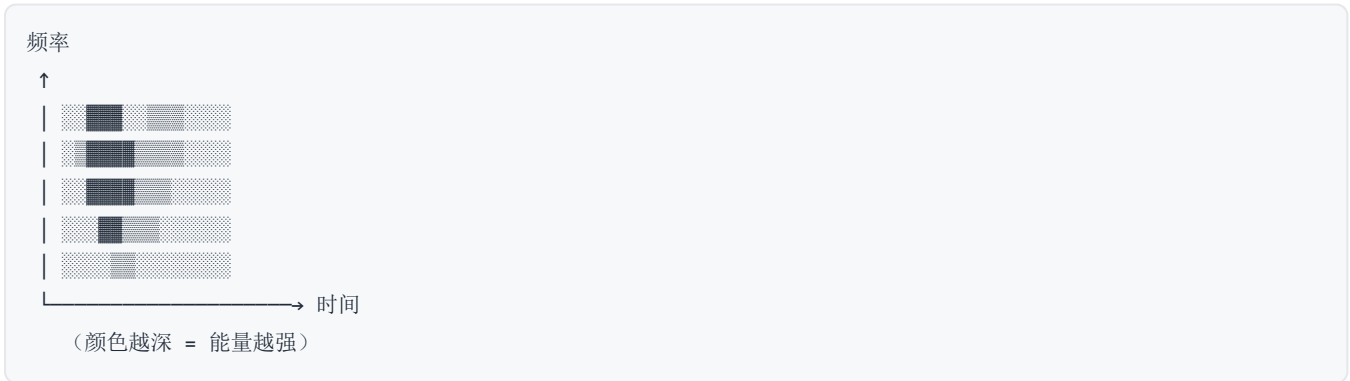
**乐器叠加的物理本质：**两种乐器同时演奏时，空气中的声波是**线性叠加的**——每个时刻的振幅直接相加。这是声源分离可行性的物理基础。

混合信号 = 乐器<sub>1</sub> + 乐器<sub>2</sub> + 乐器<sub>3</sub> + ... + 乐器<sub>n</sub>

### 1.2 频谱 (Spectrum) 与 STFT

纯波形很难看出乐器的音色差异。傅里叶变换告诉我们：**任何信号都可以分解为不同频率的正弦波之和。**

**短时傅里叶变换 (STFT)：** - 把长音频切成**重叠的小窗口** (通常 20-40ms, 如 1024 个样本点) - 对每窗做傅里叶变换, 得到该时刻的频率成分 - 所有窗口拼起来 → **频谱图 (spectrogram)**



**为什么频谱图对分离很重要？** - 不同乐器在同一时刻往往占据不同的频率区域 (钢琴低频、小提琴中高频) - 在频谱图上做分离 = 在二维"图像"上做分割——这正是深度学习最擅长的事情 - 大部分声源分离模型 (包括 SOTA 的 Demucs) 都在频谱域或"时频域"操作

### 1.3 你原文中的"时间微元"在数学上是什么？

你的直觉完全准确：

| 你的说法         | 对应技术概念                        |
|--------------|-------------------------------|
| "极短时间微元"     | STFT 窗口 (window / frame)      |
| "每段短时波形"     | 窗口内的帧信号                       |
| "各类乐器基础声波分量" | 各乐器在该帧的频谱分量                   |
| "相邻时刻不会突兀跳变" | 时序连续性约束 (temporal continuity) |

## 2. 逆问题 (Inverse Problem)

## 2.1 什么是"ill-posed" (不适定) ?

已知：混合信号  $y = x_1 + x_2 + x_3$  ( $x_1$  是钢琴,  $x_2$  是小提琴,  $x_3$  是鼓)  
求:  $x_1$ 、 $x_2$ 、 $x_3$

方程数量 (1 条混合信号) < 未知数数量 (3 条独立信号)，这是**欠定方程组**，有无穷多组解。这就是你说的“核心问题”。

## 2.2 如何变成"适定"?

加约束条件, 缩小解空间:

1. **频域稀疏性**: 各乐器在同一时频点不会同时有强能量 → 可以加稀疏约束
2. **时序连续性**: 你发现的核心约束—— $x_i(t)$  和  $x_i(t+1)$  相似
3. **乐器模板**: 钢琴的频谱结构有固定模式 ( harmonic series )
4. **学习先验**: 用大量数据训练一个神经网络, 让它“知道”钢琴听起来应该怎么样

### 3. 深度学习与声源分离

### 3.1 核心范式: Masking (掩码)

当前主流做法是把声源分离建模为**频谱掩码预测**:

混合频谱  $M \rightarrow$  神经网络  $\rightarrow$  钢琴掩码  $Mask\_piano \times M =$  钢琴频谱  
 $\rightarrow$  小提琴掩码  $Mask\_violin \times M =$  小提琴频谱  
 $\rightarrow$  鼓掩码  $Mask\_drum \times M =$  鼓频谱

每个掩码是一个和  $M$  形状相同的矩阵，值在 0~1 之间，表示"这个时频点有多少能量属于该乐器"。然后对掩码后的频谱做**逆 STFT (iSTFT)**，还原为时域波形。

### 3.2 U-Net 架构（当前最主流的基础架构）

混合频谱 (512 x T)

↓

Conv2D ↓



**为什么 U-Net 适合声源分离？** - Encoder 理解全局结构（哪个乐器在哪个时间弹了哪个音） - Decoder 恢复精细的时频结构（保留音色细节） - 跳连（skip connections）让 decoder 能看到 encoder 的原始细节

### 3.3 Demucs (Meta 的 SOTA 模型)

Demucs（尤其是 v4 Hybrid版本）是目前开源最强的声源分离模型：

- **Hybrid 架构**：同时在时域（波形）和频域（频谱图上）做处理，融合两者优势
- **Transformer 层**：在 bottleneck 位置加入 Transformer，捕捉长距离依赖
- **多尺度处理**：在不同时间分辨率上分别处理
- **4 个输出轨道**：人声、贝斯、鼓、其他

你的"时序连续性"直觉正好解释了为什么 Transformer（注意力机制能建模序列关系）对声源分离有效。

### 3.4 Masking vs. 你的构想一

| 你的构想        | 实际领域做法                                               |
|-------------|------------------------------------------------------|
| 每帧独立求解各乐器波形 | 每帧预测各乐器的频谱掩码                                         |
| 时序连续性约束     | 网络结构自带连续性（CNN 的局部连接、Transformer 的时序注意力、loss 中的平滑正则项） |
| 乐理先验校验      | 训练数据本身编码了乐理（模型在大量音乐数据上见过的统计规律）                       |

你的思路和领域主流做法惊人地一致——差异仅在于实现细节上。你从物理直觉出发推导出的框架，学术界花了十几年才摸索到相似的结构。

## 4. 关键工具速览

### 4.1 librosa 能做什么事？

```
import librosa

# 加载音频
y, sr = librosa.load('mix.wav', sr=22050) # y: numpy array, shape=(n_samples,)
```

```
# STFT → 频谱图
D = librosa.stft(y, n_fft=1024, hop_length=512) # shape=(1+n_fft//2, n_frames)
magnitude = np.abs(D) # 振幅谱
phase = np.angle(D) # 相位谱

# iSTFT → 还原波形
y_recovered = librosa.istft(magnitude * np.exp(1j * phase))
```

## 4.2 torchaudio 与 PyTorch 的集成

```
import torch
import torchaudio

# GPU 加速的 STFT
spec = torchaudio.transforms.Spectrogram(n_fft=1024)(waveform)
mel_spec = torchaudio.transforms.MelSpectrogram()(waveform)

# 可微分的 STFT → 反向传播可以穿透
```

## 4.3 数据集

| 数据集       | 规模                     | 说明                                      |
|-----------|------------------------|-----------------------------------------|
| MUSDB18   | 150 首 (训练) + 50 首 (测试) | 声源分离标准基准, 4 轨: 人声/贝斯/鼓/其他               |
| Slakh2100 | 2100 首 MIDI 合成曲        | 含 12 类乐器的 MIDI + 渲染音频, 非常适合你构想二的"逐乐器"训练 |
| MedleyDB  | 122 首多轨录音              | 真实录音, 含乐器标注                             |

## 5. 评价指标

| 指标  | 全称                           | 意义                                   |
|-----|------------------------------|--------------------------------------|
| SDR | Signal-to-Distortion Ratio   | 主要指标。分离出的乐器与真实乐器相比, 总失真 (dB) 。>10 很好 |
| SIR | Signal-to-Interference Ratio | 其他乐器漏进来的干扰成分。>15 很好                  |
| SAR | Signal-to-Artifacts Ratio    | 算法引入的伪影 (如"水声"、金属音) 。>10 很好          |

在 MUSDB18 测试集上, Demucs v4 的 SDR 约在 6-10 dB 之间 (不同乐器差异大, 人声最好分, 鼓次之, 其他最难) 。

## 6. 阅读路线

如果你是零基础:

1. **先上手体验**: 安装 `librosa` , 加载一首歌曲, 画出它的波形和频谱图
2. **理解 STFT**: 用 `librosa.stft` 和 `librosa.istft` 做一次"原样还原", 确认理解无损处理
3. **跑一个现成模型**: 安装 `demucs` 或 `spleeter` , 对一首歌做分离, 听结果
4. **读这篇论文**: [Music Source Separation in the Waveform Domain \(Demucs\)](#)——最经典的 baseline
5. **看代码**: [Meta 的 Demucs 实现](#)——架构清晰, 注释好

你完全不需要一次性掌握所有内容。**在动手实验中学习**是最快的——跑通一个模型、改一个参数、听结果变化，每次迭代都在加深理解。